אלגוריתמים 2 - הרצאה 2

21 באוקטובר 2026

גיא יער־און

סיכום זה נכתב במהלך סמס א' תשפ"ז (2027) במהלך הרצאותיהם של פרופסור אלי פורת ופרופסור משה לויינשטיין.
השימוש בסיכום באחריותכם בלבד.

0.1 Strong Duality

בפשטות, ההבדל אומר שתחת המצב האופטימלי, הפתרון הדואלי יניב בדיוק את הפתרון הפרימאלי. כלומר, החסם העליון הופך לשוויון בדיוק. כלומר, בפתרון האופטימלי: ערך פונקציית המטרה של הפתרון הדואלי יהיה שווה לערך פונקציית המטרה בפתרון הפרימאלי. וכן:

- אם בעיה אחת היא ברת ביצוע וחסומה, כלומר: פיזיבלית וחסומה, אז גם הבעיה השניה בהכרח ברת ביצוע וחסומה וערכי האופטימום שלהם שווים.
- אם בעיה אחת היא ברת ביצוע, אך לא חסומה - כלומר הערך שלה שואף לאנסוף, אזי גם הבעיה השניה איננה פיזיבלית: אין לה שום פתרון חוקי.

ובכן, תמיד מתקיים $b^T y \geq c^T x$. על מנת להוכיח דואליות חזקה, יש להראות שקיימים x, y עבורם ישנו שוויון מלא.

משפט 1. משפט הדואליות החזקה.

בתחילת תהליך אלגוריתם הסימפלקס, אנו מסתכלים על הבעיה הפרימאלית בצורתה הסטנדרטית ורוצים למקסם את

$$z = \sum_{i=1}^{n} c_i x_i$$

בנוסף, יש לנו אילוצים מסוג אי שוויון. כדי להפעיל את אלגוריתם הסימפלקס, אנו הופכים אותם למשוואות ע"י הוספת m משתנים $x_{n+1}, ..., x_{n+m}$. עד לקבלת: $x_{n+j} = b_j - \sum_{i=1}^{n} a_{j,i} x_i$ לכל $j \in [m]$.
לאחר הרצת אלגוריתם הסימפלקס, שרץ על המטריצה וכו' משוואת המטרה מקבלת צורה חדשה שכוללת קבוע מסוים יחד עם שילוב המשתנים:

$$z' = v + \sum_{i=1}^{n+m} c'_i x_i$$

הערך הקבוע v, הוא בדיוק הערך האופטימלי של פונקציית המטרה. כעת, נסתכל על הביטוי החדש של המטרה z'. נוכל לפצל אותו לשני סכומים:

$$z' = v + \sum_{i=1}^{n} c'_i x_i + \sum_{j=1}^{m} c'_{n+j} x_{n+j}$$

ובכן, אנו יודעים כי $x_{n+j} = b_j - \sum_{i=1}^{n} a_{j,i} x_i$ ולכן נוכל להציב:

$$z' = v + \sum_{i=1}^{n} c'_i x_i + \sum_{j=1}^{m} c'_{n+j} \cdot \left(b_j - \sum_{i=1}^{n} a_{j,i} x_i \right)$$

נרצה להשוות את z' לz. לשם כך, נפתח סוגריים, נסדר את האיברים ונקבץ את הקבועים ונקבל:

$$z' = v + \sum_{j=1}^{m} c'_{n+j} b_j + \sum_{i=1}^{n} \left(c'_i - \sum_{j=1}^{m} c'_{n+j} a_{j,i} \right) x_i$$

זו פונקציה המורכבת מאיבר חופשי קבוע יחד עם סכום מקדמים כפול המשתנים המקוריים. יש לשים לב: z, z' הן אותה הפונקציה על כל הצבה חוקית של המשתנים בבעיה. לכן,

$$\forall x_1, ..., x_n : z(x_1, ..., x_n) = z'(x_1, ..., x_n)$$

נזכר כי $z = 0 + \sum_{i=1}^{n} c_i x_i$ וכיוון שz, z' הן שוות לכל וקטור, האיברים החופשיים שלהם חייבים להיות שווים גם כן. ובפרט, כל מקדם בהם חייב להיות שווה. לכן:

- הקבוע שווה ולכן:

$$0 = v + \sum_{j=1}^{m} c'_{n+j} b_j \Longrightarrow v = -\sum_{j=1}^{m} c'_{n+j} b_j$$

- השוואת מקדמים: לכל $1 \leq i \leq n$ נקבל כי:

$$c_i = c'_i - \sum_{j=1}^{m} c'_{n+j} a_{j,i}$$

נשים לב כי בבעיה הדואלית אנו מחפשים משתנים y_j שכופלים את b_j בפונקציית המטרה, אנו מצאנו קשר כל" ל עבור $v = -\sum_{j=1}^{m} c'_{n+j} b_j$. לכן, נגדיר את משתני הדואל להיות המקדמים הללו. כלומר:

$$y_j = -c'_{n+j}$$

נראה כי:

$$b^T y = \sum_{j=1}^{m} b_j y_j = \sum_{j=1}^{m} b_j \cdot (-c'_{n+j}) = v$$

לכן, קיבלנו כי ערך פונקציית המטרה הדואלית שווה בדיוק לv, הפתרון האופטימלי הפרימאלי. לכן קיבלנו את השוויון שרצינו. אך עוד לא סיימנו, עלינו להוכיח כי וקטור הy שהוגדר הוא פתרון פיזיבלי לבעיה הדואלית. כדי שפתרון יהיה חוקי בבעיה הדואלית עליו לקיים:

1. $A^T y \geq c$
2. $\forall j : y_j \geq 0$

קודם ראינו כי $c_i = c'_i - \sum_{j=1}^{m} c'_{n+j} a_{j,i}$, אם נסדר נקבל $c_i - c'_i = \sum_{j=1}^{m} -c'_{n+j} a_{j,i}$ כלומר:

$$c_i - c'_i = \sum_{j=1}^{m} y_j a_{j,i}$$

כיוון שהגענו לטבלה האופטימלית בהרצת הסימפלקס, כל המקדמים בשורת המטרה מקיימים כי $c'_i \leq 0$ (שיפוע שלילי). לכן בהכרח $c_i \leq c_i - c'_i$ ומכאן נקבל: $c_i \leq \sum_{j=1}^{m} y_j a_{j,i}$ ולכן $A^T y \geq c$.
אותו חוק אופטימליות של הסימפלקס פועל על משתני ההשלמה, $0 \leq c'_{n+j}$. לכן, בהכרח $y_j = -c'_{n+j} \geq 0$ לכל j.
סה" כ מצאנו פתרון אפשרי, פיזיבלי, לבעיה הדואלית שערכו הפונקציונלי שווה בדיוק לפתרון האופטימלי של הבעיה הפרימאלית. לכן, *Primal=Dual*.

■

כמה שאלות שעולות:

- מדוע z, z' תמיד שווים לכל הצבה וניתן להשוות בניהם? כיוון שz' איננה פונקציה חדשה שהמצאנו, אלא התקבלה במהלך אלגוריתם הסימפלקס. במהלך האלגוריתם אנו מבצעים פעולות שורה אלמנטריות על מערכת המשוואות של האילוצים שלנו. כאשר

אנו מציבים את משתני ההשלמה לתוך z', אנו מבצעים מניפולציה אלגברית. מבחינה מתמטית, מדובר באותה פונקציה, כתובה באופן שונה.

- לפי הדואליות החלשה, הפתרון הדואלי מהווה חסם עליון לפרימאלי. לפי הדואליות החזקה, באופטימום, ורק בו, יתקיים שוויון בין ערכי הפרימאל והדואלי.
- למה אנו צריכים את הבעיה הדואלית? בעולם האמיתי, לעיתים בעיה פרימאלית כוללת מליוני אילוצים אך רק מעט משתנים. במעבר לבעיה הדואלית, האילוצים הופכים למשתנים והמשתנים לאילוצים. במצבים כאלו, להריץ תוכנה שתפתור את הדואלית יעיל יותר מלפתור את הבעיה המקורית.
- אם הבעיה פיזיבלית וחסומה, אלגוריתם הסימפלקס שרץ על הבעיה הדואלית יניב את הפתרון האופטימלי.

0.2 אלגוריתמי קירוב ותכנות לינארי

נחזור לדון על VC, זו בעיה NPC שאינה ניתנת לפתרון פולינומי. גם הגרסה שלה ע"י תכנות בשלמים לא ניתנת לפתרון, כיוון ש$IP \in NPH$ גם כן. אם נשנה את הגרסה, כך שלא נדרוש שהמשתנים הבינאריים יהיו אפס או אחד בלבד, נקבל:

$$\min \sum_{v \in V} x_v \ \forall (u,v) \in E : x_v + u_v \geq 1 \wedge \forall v : 0 \leq x_v \leq 1$$

האילוץ הופך את הבעיה לניתנת לפתרון ע"י תכנות לינארי. כעת, הבעיה ניתנת לפתרון יעיל באמצעות LP רגיל. כיוון שLP מאפשר יותר אפשרויות, הערך המינימלי שתמיד ימצא תמיד יהיה קטן או שווה לערך המינימלי של IP ולכן יתקיים $LP \leq IP$. זה יאפשר לנו לחשב חסם תחתון עבור VC.

נניח משולש: u,v,w. הרצת LP עליו יניב $x_v = x_u = x_w = \frac{1}{2}$. כלומר, $LP = 1.5$. אך בפועל, אי אפשר לקחת חצי צומת - לכן כדי לכסות משולש יש לבחור לפחות 2 צמתים שלמים. לכן הפתרון האופטימלי הוא $IP = OPT = 2$. כלומר, אנחנו רוצים למצוא פתרון שאיננו בהכרח אופטימלי, אך הוא טוב.

0.2.1 אלגוריתם קירוב לVC עם תכנות לינארי

האלגוריתם פשוט למדי: נריץ LP על הבעיה שאינה בשלמים, ונגדיר את הקבוצה הבאה:

$$S = \{v | x_v \geq \frac{1}{2}\}$$

משפט 2. מדובר בכיסוי חוקי, עם יחס קירוב $S \leq 2LP \leq 2OPT$

הוכחה: ראשית, מדובר בכיסוי חוקי: שכן, לכל קשת מוכרח כי מתקיים $x_u + x_v \geq 1$.

נניח בשלילה כי קיימת קשת $e = (u,v)$ שלא מכוסה ע"י הכיסוי. אזי, $x_u, x_v < \frac{1}{2}$ ולכן $x_u + x_v < 1$ בסתירה לאילוץ ולנכונות LP. לכן, בהכרח אחד מהצמתים מכוסה ע"י הכיסוי.

כעת, נוכיח את יחס הקירוב. גודל הקבוצה $|S|$ שלקחנו הוא פי' הצמתים שלקחנו. לקחנו רק צמתים שהערך שלהם הוא לפחות חצי, ולכן אם נעגל אותם לאחד, אנו לכל היותר מכפילים את התרומה שלהם לפונקציית המטרה פי 2. לכן, גודל הקבוצה S יהיה לכל היותר פעמיים הגודל של ערך הLP. שכן, מתמטית:

$$|S| = |\{x_v | x_v \geq \frac{1}{2}\}| \leq 2LP \leq 2IP = 2OPT$$

■

הערה חשובה. העולם של דואליות מגיע בשביל למצוא חסם לפתרון. נניח ואנחנו בבעית מקסימום, אזי המטרה שלנו היא למצוא חסם עליון לבעיה שלנו. כלומר ערך a שפונקציית המטרה שלנו לא יכולה לעבור. לכן אנחנו יודעים שבמקרה זה ערך פונקציית המטרה של הבעיה הדואלית חוסם את ערך פונקציית המטרה של הבעיה הפרימאלית כלומר $\text{Primal} \leq \text{Dual}$.

לעומת זאת, בבעיות מינימיזציה המטרה שלנו ליצור בעיה דואלית שחוסמת לנו מלמטה - כלומר ערך a' אשר תמיד מתקיים כי a גדול ממנו. כלומר במקרה זה של בעיות מינימיזציה אזי $\text{Dual} \leq \text{Primal}$.

Set Cover 0.2.2

הגדרה 3. בהינתן עולם U ואוסף תתי קבוצות $S_1, ..., S_\ell \subseteq U$, בעיית Set Cover מוגדרת להיות מציאת המס' המינימלי של קבוצות k מתוך האוסף כך שאיחודן יכסה את היקום. כלומר,

$$\bigcup_{i \in I} S_i = U$$

כך ש$I \subseteq \{1, ..., \ell\}$ וגם $|I| = k$.

ע"י הגדרה לפי ניסוח בתכנות לינארי, לכל קבוצה S_i נגדיר משתנה בינארי $x_i = 1$ אמ"מ S_i נלקחה לכיסוי.
נוכל להגדיר את הבעיה:

$$\min \sum_{i=1}^{\ell} x_i \text{ s.t } \forall a \in U : \sum_{i:a \in S_i} x_i \geq 1$$

כלומר, לכל איבר בעולם $a \in U$ סכום משתני ההחלטה x_i עבור הקבוצות המכילות את a חייב להיות גדול או שווה ל1. על מנת לפתור את הבעיה בתכנות לינארי, במקום $x_i \in \{0, 1\}$ משנים את האילוץ ל$x_i \geq 0$.

משפט 4. קיים אלגוריתם f־קירוב ל*Set Cover*.

כעת, לאחר שפתרנו את הLP קיבלנו ערכים שבריים. אנו צריכים להחליט אילו קבוצות באמת לקחת לכיסוי. יהי f פרמטר שמוגדר להיות המס' המקסימלי של קבוצות שאיבר כלשהו בU שייך אליהן בו זמנית. כלומר: אם האיבר הכי נפוץ נמצא ב3 קבוצות אזי $f = 3$. פורמלית:

$$f = \max_{a \in U} |\{i | a \in S_i\}|$$

נגדיר: $I = \{i | x_i \geq \frac{1}{f}\}$. נטען כי קבוצה זו בהכרח מכסה את כל האיברים. יהי איבר כלשהו $a \in U$, סכום הערכים של הקבוצות המכילות אותו הוא לפחות 1 (שכן הוא שייך לפחות לקבוצה אחת). הרי, ישנן לפחות f קבוצות כנ"ל, ולכן לפי עקרון שובך היונים קיימת לפחות קבוצה אחת מתוכן אשר קיבלה ערך x_i גדול־שווה מ$\frac{1}{f}$. לכן, בהכרח הקבוצה הנ"ל תבחר לאלגוריתם שלנו: ולכן a יכוסה.
כיוון שבחרנו רק קבוצות המקיימות $x_i \geq \frac{1}{f}$, אם נעגל את הערך שלהן ל1 הרי שנכפיל את הכמות הכללית לכל היותר פי f. לכן גודל הכיסוי שלנו יהיה לכל היותר $f \cdot LP$)כלומר, לכל היותר הכפלה פי f מהכמות שמקבלים בLP: הרי קיבלנו גודל ואז הניתוח מתייחס לאחר ניפוח פי f). מכאן:

$$|\text{Set-Cover}| = f \cdot LP \leq f \cdot IP \leq f \cdot OPT$$

כנדרש.

■

èôùî 5. קיים אלגוריתם הסתברותי שפותר את *Set-Cover* בהסתברות גבוהה.
הוכחה: כעת, במקום להשתמש בx_i שהניב האלגוריתם כערך חותך, נשתמש בו כהסתברות. האלגוריתם יוגדר באופן הבא:
נטיל מטבע מוטה לכל קבוצה S_i. הקבוצה תבחר לכיסוי בהסתברות שערכה x_i (הערך שקיבלנו מהLP). נראה כי לפי לינאריות התוחלת, תוחלת גודל הקבוצה הינה:

$$\mathbb{E}[|I|] = \sum_{i=1}^{\ell} \mathbb{E}[x_i] = \sum_{i=1}^{\ell} x_i = LP$$

כלומר, אכן בממוצע נכסה את כולם אך אין לנו ודאות שאכן כל איבר יכוסה. לכן, נרצה לחשב את ההסתברות לכישלון, ולהוכיח שהיא קטנה: זה מספיק טוב עבור אלגוריתם הסתברותי. ובכן, איבר לא נבחר אם"מ אף אחת מהקבוצות לא בחרה אותו. המאורע לבחירה של קבוצה כ"ת לקבוצות שונות ולכן נקבל כי

$$Pr[\ i\ Failed] = \prod_{i:a\in S_i}(1-x_i) \leq \prod_{i:a\in S_i} e^{-x_i} = e^{-\sum_{i:a\in S_i} x_i} \leq_* e^{-1}$$

שכן $*$ נכון כי $\sum_{i:a\in S_i} x_i \geq 1$. כעת, נרצה להגביר את ההסתברות. נגדיל באופן מלאכותי את ההסתברות לבחור כל קבוצה. ונבחר את הקבוצה S_i בהסתברות המוגדלת $p_i = x_i \cdot (ln(n)+1)$. באשר, $n = |U|$. המחיר בכך, הוא שתוחלת כמות הקבוצות שבחרנו גדלה להיות $\mathbb{E}[|I|] = (\ln(n)+1)\cdot LP$. כעת, ההסתברות עבור איבר בודד לכישלון הופכת להיות:

$$e^{-(\ln(n)+1)} = \frac{1}{n\cdot e}$$

מכאן:

$$\Pr[failed] = \bigcup_{i=1}^{n}\Pr[i\ failed] \leq \sum_{i=1}^{n}\Pr[i\ failed] = n\cdot\frac{1}{ne} = \frac{1}{e}$$

כלומר, הסתברות הכישלון הכוללת של האלגוריתם היא e^{-1}, והאלגוריתם מניב כיסוי בגודל לכל היותר $\ln(n)$ כפול הפתרון האופטימלי.
■

0.2.3 הגרסה הדואלית של Set Cover

כעת, נרצה לעבור לגרסה הדואלית ולשם כך לכל אילוץ בבעיה הפרימאלית (כלומר לכל איבר $a \in U$) נגדיר משתנה דואלי חדש y_a. פונקציית המטרה משתנה ממינימיזציה למקסימיזציה, ומטרתה למקסם את סכום המשתנים הדואלים. וכן, לכל קבוצה $1 \leq i \leq \ell$ סכום המשתנים הדואליים עבור כל האיברים a ששייכים לקבוצה, חייב להיות קטן או שווה לאחד. סה"כ נקבל:

$$\max\left(\sum_{a=1}^{n} y_a\right)\ \text{s.t}\ \forall i\in\{1,...,\ell\}: \sum_{a\in S_i} y_a \leq 1\ ,\forall 1\leq a\leq n: y_a \geq 0$$

כלומר, האילוץ מוגדר עבור כל קבוצה בנפרד - סכום המשתנים של האיברים בקבוצה לא יכול לעבור את אחד.

משפט 6. תהי $D \subseteq U$ תת קבוצה המקיימת את התנאי $|D \cap S_i| \leq t$ לכל קבוצה S_i באוסף. אזי, גודל הכיסוי האופטימלי יקיים $Opt \geq \frac{|D|}{t}$.

הוכחה: למעשה, נראה כי קיימת הצבה חוקית למשתני הבעיה הדואלית המניבה ערך פונקציית מטרה של $\frac{|D|}{t}$. נגדיר את משתני הבעיה הדואלית y_a לכל איבר $a \in U$ באופן הבא: עבור כל איבר שלא שייך לD, נציב $y_a = 0$. לכל איבר $a \in D$ נציב $y_a = \frac{1}{t}$. ראשית, נוכיח כי הצבה הנ"ל מהווה פתרון חוקי לבעיה הדואלית. על מנת שהפתרון יהיה חוקי, עליו לקיים את האילוץ $\sum_{a\in S_i} y_a \leq 1$ לכל קבוצה S_i. נראה כי:

$$\sum_{a\in S_i} y_a = \sum_{a\in S_i\setminus D} y_a + \sum_{a\in S_i\cap D} y_a = \sum_{a\in s_i\setminus D} 0 + \sum_{a\in S_i\cap D}\frac{1}{t} \leq 0 + t\cdot\frac{1}{t} = 1$$

לכן, ההצבה הנ"ל מהווה פתרון חוקי לבעיה. כעת, נחשב את ערך פונקציית המטרה עבור הצבה זו. נזכר כי הגדרנו ערך ששונה מאפס רק עבור איברי הקבוצה D לכן נקבל:

$$\sum_{a=1}^{n} y_a = \sum_{a\in D} y_a = \sum_{a\in D}\frac{1}{t} = \frac{|D|}{t}$$

לפי משפט הדואליות החלשה, ערך פונקציית המטרה עבור כל פתרון דואלי חוקי חסום מלמעלה ע"י ערך פונקציית המטרה של כל פתרון פרימלי חוקי. הרי כי Opt הינו העלות הפרימלית האופטימלית, והצגנו פתרון דואלי חוקי. לכן יתקיים: $Opt \geq \frac{|D|}{t}$. כנדרש.

הערה 7. מה אומר המשפט העיל למעשה? מצאנו חסם תחתון לפתרון האופטימלי של Set Cover, מבלי לפתור את הבעיה הקשה בעצמה. כיוון שקשה למצוא את הפתרון האופטימלי, ניתן להעזר בחסם לבנות קבוצה D מתאימה, לחשב את הערך $\frac{|D|}{t}$ ולקבל חסם תחתון לפתרון.

הערה 8. מדוע הבעיה הדואלית הוגדרה כך? בבעיה הפרימאלית, יש לנו משתנה x_i לכל קבוצה S_i ואנו מנסים לצמצם את סכומם. בבעיה הדואלית, המשתנים שלנו הם y_a והם מוגדרים לכל איבר ביקום ואנו מנסים למקסם את סכומם. ישנה סימטריה מלאה: איברים הופכים לאילוצים, קבוצות הופכות למשתנים.

בפרימאל, יש אילוץ לכל $a \in U$ שדורש כי הקבוצות שמכילות אותו יכסו אותו לפחות באחד: $\sum x_i \geq 1$. האילוץ הזה בפרימאל מוליד בדואל משתנה y_a.

בדואל, יש אילוץ לכל קבוצה S_i - אשר דורש כי סכום המשקולות בו לא יעלה על 1. אילוץ זה בודק את הקשר ההדדי בין קבוצה לאיברים שמרכיבים אותה.

הפרימיאל, מנסה למצוא את המינימום - לשלם כמה שפחות עבור כיסוי. הדואל - שואף למצוא את המקסימום: להרוויח כמה שיותר ע"י תמחור נכון של האיברים - משפט הדואליות החלשה מבטיח כי כל פתרון חוקי של הדואלי לעולם לא יעקוף את הפרימיאל.

Greedy Set Cover 0.2.4

נציג אלגוריתם חמדן שפותר את Set-Cover ביחס קירוב לוגריתמי בגודל העולם.

בתחילה, אנו מגדירים קבוצה D שמכסה את כל היקום בעת אתחול האלגוריתם. כל עוד יש איברים שלא כוסו - אנחנו בוחרים באופן חמדני את הקבוצה S_i שמכילה את המס' הגדול ביותר של איברים שעוד לא כוסו, מוסיפים לI (אוסף האינדקסים של הקבוצות שנבחרו) את אינדקס הקבוצה, ומעדכנים את D להיות הקבוצה פחות האיברים שכעת כבר כוסו. כך מתקדמים, כל עוד $D \neq \emptyset$.

Algorithm: Greedy Set Cover

Require: Universe U with $|U| = n$, collection of sets $S_1, S_2, \ldots, S_\ell$ such that $\bigcup_i S_i = U$
Ensure: A collection of indices I such that $\bigcup_{i \in I} S_i = U$

1. $D \leftarrow U$ (the elements not yet covered)
2. $I \leftarrow \emptyset$ (the chosen set indices)
3. While $D \neq \emptyset$:
   (a) $i \leftarrow \arg\max_i |D \cap S_i|$ (the set covering the most remaining elements)
   (b) $I \leftarrow I \cup \{i\}$
   (c) $t \leftarrow |D \cap S_i|$ (how many new elements this set covers)
   (d) $D \leftarrow D \setminus S_i$
4. **return** I

למען ניתוח האלגוריתם, נעזר במשפט שהוכח קודם. לכל תת קבוצה $D' \subseteq U$ בה כל קבוצה חותכת את D בלא יותר מt איברים, מתקיים $Opt \geq \frac{|D'|}{t}$. כעת, נראה כיצד האלגוריתם החמדן מנצל את המשפט הנ"ל איטרציה אחר איטרציה. נניח כי האלגוריתם בחר בסך הכל k קבוצות באיטרציות שונות. נסמן את הקבוצות שנבחרו $S_{i_1}, ..., S_{i_k}$ ואת גודל החיתוך שלהן עם האיברים שטרם כוסו באותם רגעים ב$t_1, ..., t_k$.

בתחילת הדרך, מתקיים $D = U$. הקבוצה הראשונה שנבחרה, S_{i_1} מכסה מס' איברים שנגדירו כ$t_1 = |S_{i_1}|$. כלומר האיברים שנבחרו הם $a_1^1, ..., a_{t_1}^1$. נפעיל את משפט החסם העליון על שלב זה, והרי כי מתקיים:

$$Opt \geq \frac{n}{t_1} \Longrightarrow \frac{Opt}{n} \geq \frac{1}{t_1}$$

נשים לב כי:

$$\frac{1}{t_1} + \frac{1}{t_1} + ... + \frac{1}{t_1} = 1 \leq \frac{Opt}{n} + ... + \frac{Opt}{n}$$

באשר מספר האיברים בסכימה משמאל הוא t_1. כלומר, $\frac{Opt}{n} \geq \frac{1}{t_1}$ עבור האיברים הראשונים שנוספו.
כעת, לאחר שבחרנו את הקבוצה הראשונה נותרו ביקום $n - t_1$ איברים. הקבוצה השנייה שנבחרה מקיימת $|D \cap S_{i_2}| = t_2$.
האלגוריתם החמדן בוחר בכל רגע את הקבוצה שמכסה כמה שיותר מהאיברים הנותרים, לכן בהכרח גודל החיתוך של כל קבוצה בהמשך לא יכול לעבור את t_2. מכאן נקבל יחס $t_1 \geq t_2 \geq t_3 \geq ...$. מכאן על הקבוצה בגודל $n - t_1$ נקבל:

$$Opt \geq \frac{n - t_1}{t_2} \Longrightarrow \frac{Opt}{n - t_1} \geq \frac{1}{t_2}$$

באופן כללי, עבור האיטרציה הj מס' האיברים שנותרו לכיסוי הינו $|U| - \sum_{i=1}^{j-1} t_i$. לכן החסם התחתון יקיים

$$Opt \geq \frac{n - \sum_{i=1}^{j-1} t_i}{t_j}$$

נסכום את כל אי השוויונות שקיבלנו לאורך האיטרציות,הרי בכל שלב המכנה הוא מהצורה $n - \sum_{i=1}^{j-1} t_i$. מכנה זה הוא מס' האיברים שנותרו לכיסוי. סה"כ לאורך האיטרציות עוברים על כל הערכים במקרה הגרוע ביותר (בו ההתקדמות היא הכי איטית - בכל שלב אנחנו מוסיפים רק איבר אחד).
סך הקבוצות שנבחרו הוא בדיוק k, כל איבר מקבל חסם שמורכב משברים מהצורה $\frac{1}{t_j}$. מכאן נקבל:

$$k = \sum_{i=1}^{k} \sum_{a \in S_i \cap D} \frac{1}{t_i} \leq \sum_{i=1}^{k} \sum_{a \in S_i \cap D} \frac{\text{Opt}}{n - \sum_{j=1}^{i-1} t_i} =\leq \text{Opt} \cdot \left(\frac{1}{n} + \frac{1}{n-1} + ... + \frac{1}{3} + \frac{1}{2} + \frac{1}{1} \right) = \text{Opt} \cdot \sum_{j=1}^{n} \frac{1}{j} = \ln(n) \cdot \text{Opt}$$

לכן מס' הקבוצות שהאלגוריתם יניב הוא לכל היותר פקטור לוגריתמי במס' האיברים כפול הפתרון האופטימלי.

Max Flow Min Cut 0.3

הגדרה 9. רשת זרימה היא גרף מכוון בו קיים קודקוד מקור s וקודקוד יעד t, כך שקיימת פונקציית קיבולת לכל קשת בגרף. יהי x_{r_1,r_2} משתנה המייצג את כמות הזרימה שעוברת בקשת (r_1, r_2), ויהי $w(r_1, r_2)$ משתנה שמייצג את הקיבולת המקסימלית המותרת על הקשת (r_1, r_2).

נגדיר את בעיית זרימת המקסימום בצורה של תכנות לינארי:

$$\max \sum_{w \in \Gamma(s)} x_{s,w} \text{ s.t} \begin{cases} \forall w \in V \backslash \{s,t\} : \sum_{r \in \Gamma(w)} x_{w,r} - \sum_{r \in \Gamma(w)} x_{r,w} = 0 \\ \forall (r_1, r_2) \in E : x_{r_1,r_2} \leq w_{r_1,r_2} \\ \forall (r_1, r_2) : x_{r_1,r_2} \geq 0 \end{cases}$$

באשר האילוצים הם חוקי הזרימה: שימור הזרימה, אילוצי הקיבולת ואי השליליות על הזרימה.

1 Succinct Data Structures: מבני נתונים תמציתיים

באופן כללי, כאשר נדבר על מבני נתונים תמציתיים המטרה שלנו היא במקום לבזבז במבנה נתונים זכרון רגיל - לייצג את הנתונים במינימום מקום האפשרי לכך.

Rank Problem 1.1

הבעיה מוגדרת באופן הבא:

הגדרה 10. יהי מערך $A=(a_1,...,a_n)$ כך שלכל $i\in[n]$ מתקיים $a_i\in\{0,1\}$. בהינתן אינדקס i, עלינו להחזיר כמה אחדות ישנם במערך מתחילתו ועד לאינדקס i. כלומר:

$$\mathrm{Rank}\,(i)=\sum_{j=0}^{i}A\,[j]$$

על מנת לפתור את הבעיה, נוכל להציג כמה פתרונות. חלקם, נאיביים.

1.1.1 אלגוריתם ראשון

נוכל לבצע את הלולאה לסכימה בעלות לכל היותר $\mathcal{O}\,(n)$, ללא זיכרון נוסף וללא עיבוד מקדים.

1.1.2 אלגוריתם שני

לחשב את $\mathrm{Rank}(i)=\mathrm{Rank}\,(i-1)+A\,[i]$. נבצע זאת כך באמצעות עיבוד מקדים ע"י תכנות דינמי בעלות $\mathcal{O}\,(n)$ וכעת זמן השאילתא יהיה $\mathcal{O}\,(1)$. אך, נשים לב כי סיבוכיות הזיכרון במקרה זה היא $\mathcal{O}\,(nlogn)$. מדוע? עלינו לשמור מערך Rank. לכל i מתקיים $\mathrm{Rank}\,(i)\leq n$ ולכן צריך $\log_2(n)$ ביטים לייצוג. לכן מס' הביטים לייצוג הוא לכל היותר

$$n\log_2(n)$$

1.1.3 אלגוריתם שלישי

במקום לשמור תשובה לכל אינדקס ואינדקס, שזה יקר מאוד, או לא לשמור כלום כמו באפשרות הנאיבית - נבצע חלוקה לבלוקים. נחלק את המערך הגדול לבלוקים באורך t, ונשמור תשובה מראש רק עבור תחילת כל בלוק. ישנם סה"כ $\frac{n}{t}$ בלוקים ולכן נפח הזיכרון שנצטרך לשמור הוא $\frac{n}{t}\log_2(n)$ ביטים.

כיצד מתבצעת שאילתא בפתרון שכזה?

- קפיצה לבלוק הרלוונטי בזמן $\mathcal{O}\,(1)$
- בתוך הבלוק עצמו, ספירה ידנית של האחדות בזמן $\mathcal{O}\,(t)$.

נשים לב כי אם נקח את $t=c\log_2(n)$ באשר $c\geq 1$ קבוע, נקבל כי זמן שאילתא הוא $\mathcal{O}\,(\log(n))$ ואילו סיבוכיות הזיכרון הינה $\frac{n}{c\log_2(n)}\cdot\log_2(n)=\frac{n}{c}=\mathcal{O}\,(n)$. כלומר, הגענו לסיבוכיות זיכרון לינארית.

1.1.4 אלגוריתם רביעי

בפתרון הקודם, היינו צריכים לשלם בזמן השאילתא $\mathcal{O}\,(\log(n))$ כיוון שהיינו צריכים לספור ידנית בתוך בלוק בגודל t. הרעיון כעת

- נפעיל את אותו הטריק פעם נוספת בתוך כל בלוק.

כיצד המבנה כעת יהיה בנוי?

- בלוקים גדולים, אשר מחלקים את המערך לבלוקים בגודל t. לכל בלוק שכזה שומרים את מס' האחדות המצטבר עד לתחילתו, ישנם $\frac{n}{t}$ בלוקים כאלו ולכן סיבוכיות הזיכרון בשלב זה הינה $\frac{n}{t}\log_2(n)$ ביטים.
- תת בלוקים: בכל בלוק גדול בגודל t, מחלקים אותו לחתיכות קטנות יותר. נשמור את מס' האחדות יחסית לתחילת הבלוק הגדול - על מנת שהערכים יהיו קטנים ולא ידרשו הרבה ביטים. הערך המקסימלי שנצטרך לשמור הוא t, לכן כל ערך כזה דורש לכל היותר $\log_2(t)$ ביטים.

סה"כ יש n איברים מחולקים לתתי בלוקים, ולכן תוספת הזיכרון לחלק זה היא $n\log_2(t)$.

כעת:

- זמן טרום העיבוד: $\mathcal{O}\,(n)$ עבור מעבר על המערך ובניית הטבלאות של חישובי הבלוקיסג.
- שאילתא: $\mathcal{O}\,(1)$. מדוע?

על מנת לחשב $\mathrm{Rank}\,(i)$ מבצעים:

1. לוקחים את הערך של הבלוק הגדול בו נמצא האינדקס i, בעלות $\mathcal{O}\,(1)$.
2. מוסיפים לו את הערך של תת הבלוק התואם לi בבלוק הגדול, בעלות $\mathcal{O}\,(1)$.

- סה"כ סיבוכיות הזיכרון הינה:

$$\frac{n}{t}\log_2(n)+n\log_2(t)$$

אם נקח $t=c\log_2(n)$ עבור $c\geq 1$ קבוע, נקבל כי סיבוכיות הזיכרון הינה:

$$\frac{n}{c}+n\log_2(c\log_2(n))=\mathcal{O}(n\log(\log(n)))$$

כלומר, קיבלנו מבנה נתונים הנותן שאילתות בזמן $\mathcal{O}(1)$ עם עיבוד מקדים בעלות לינארית, ותוספת הזיכרון קטנה מאוד ביחס למינימום התאורטי.

1.1.5 אלגוריתם חמישי

אם האלגוריתם הקודם היה כל כך טוב, למה לעצור בשתי קריאות? נוכל להמשיך ולחלק את הבלוקים שוב ושוב, בתוך עצמם. נבצע רמות של בלוקים באופן רקורסיבי.

נקח את הבלוק, אותו נחלק לתתי בלוקים, וכל אחד מתתי הבלוקים מחולק שוב לתתי בלוקים. כך עד שמגיעים לגודל מינימלי.

כעת, זמן טרום העיבוד נותר $\mathcal{O}(n)$. זמן השאילתא יהיה כעת $\mathcal{O}(\log^*(n))$, כמס' הפעמים שיש להפעיל לוגריתם עד שמגיעים לאחד: שכן עלינו לחבר את כל התוצאות הללו מהמבנים השונים. תוספת הזיכרון תרד בצורה משמעותית - כעת סיבוכיות הזיכרון תהיה $\mathcal{O}(n\log^*(n))$ בלבד.

1.1.6 אלגוריתם שישי

אם כן, במקום לבצע רקורסיה או לחלק לתתי בלוקים עמוקים. נסתכל על גודל בלוק קטן מספיק, נקרא לו t. כמה אפשרויות שונות יכולות להיות לבלוק בינארי בגודל t? כיוון שהבלוק מורכב מאפסים ואחדות, יש סה"כ 2^t אפשרויות. אם t קטן מספיק, 2^t הוא קטן מספיק על מנת שנוכל להכין טבלה מראש לכל האפשרויות של בלוקים בגודל t. כיצד זה עובד בפועל?

- הבלוקים הגדולים: עדיין נשמור מערך חיצוני שיתן את מס' האחדות עד תחילת כל בלוק בעלות זיכרון $\frac{n}{t}\log_2(n)$.
- הטבלה האוניברסלית: לכל מחרוזת בינארית אפשרית באורך t, נחשב מראש טבלה שתניב את הRank הפנימי לכל מיקום בתוך הבלוק. כל תשובה בתוך הטבלה דורשת $\log_2(t+1)$ ביטים לכל היותר. לכן גודל הטבלה הכולל הוא $2^t\cdot\log_2(t+1)$. עבור t הבלוקים נקבל שחלק זה צורך זיכרון של $2^t\cdot t\cdot\log_2(t+1)$.

סיבוכיות המבנה היא:

$$2^t\cdot t\cdot\log_2(t+1)+\frac{n}{t}\log_2(n)$$

עבור $t=\frac{1}{2}\log_2(n)$ נקבל:

$$2^{\frac{1}{2}\log_2(n)}\cdot\frac{1}{2}\log_2(n)\log_2\left(\frac{1}{2}\log_2(n)+1\right)+\frac{2n}{\log_2(n)}\log_2(n)=2n+o(n)$$

כלומר, סיבוכיות הזיכרון היא לינארית וסיבוכיות השאילתא במקרה זה היא בדיוק $\mathcal{O}(1)$.

1.2 Select Problem

הבעיה מוגדרת באופן הבא:

הגדרה 11. נתון מערך בינארי $A\in\{0,1\}^n$. באשר הוא מעובד מראש. בהינתן $i\in[n]$ עליך למצוא x כך ש:

$$\sum_{j=1}^{x}A[j]=i$$

נשים לב כי נרצה את הx הקטן ביותר שמקיים זאת. הסכום נשאר הרי i גם אם יש אפסים לאחר מכן.

1.2.1 פתרון ראשון

סריקה: נבצע סריקה של המערך ובאופן נאיבי נפתור את הבעיה. זמן השאילתא $\mathcal{O}(n)$.

1.2.2 פתרון שני

בעיבוד מקדים עוברים פעם אחת על המערך בזמן $\mathcal{O}(n)$ ושומרים מערך P כך ש$P[i]$ הוא המיקום הראשון של x הנ"ל שיש לחפש. זמן הסריקה הוא לינארי כמובן.

לאחר מכן, השאילתא היא קריאה אחת מהמערך - זמן $\mathcal{O}(1)$

המחיר הוא במקום - יתכנו עד n אחדים ולכן כל מספר ידרוש $\mathcal{O}(\log_2(n))$ ביטים - לכן סה"כ $\mathcal{O}(n\log(n))$ ביטים. כעת, נרצה להוריד את המקום אבל גם לשמור על זמן שאילתא מהיר יחסית.

1.2.3 פתרון שלישי

כעת, במקום לשמור את המיקום של כל אחד שומרים רק את המיקום של האחד שהסכום עד אליו $t, 2t, 3t$ וכו'.

כעת יש לכל היותר $\frac{n}{t}$ דגימות: כל אחת בעלות $\mathcal{O}(\log_2(n))$ ביטים ולכן סה"כ המיקום הוא $\mathcal{O}\left(\frac{n}{t}\cdot\log_2(n)\right)$.

שאילתא: על מנת למצוא את הi, נחשב $k=\frac{i}{t}$ בערך שלם תחתון. נקפוץ לדגימה הk, ונמשיך לספור משם עוד $i \mod t$ אחדות. נשים לב כי יתכן שהסכימה תשאר רק בתחום בין הדגימה הk לדגימה ה$k+1$. זהו קטע שאורכו d_i. לכן, זמן השאילתא $\mathcal{O}(d_i)$. מה הבעיה? אין לנו חסם טוב על d_i.

לכן, נשתמש בשיטת **קל כבד.**

נבחר סף c ונטפל בכל קטע לפי האורך שלו:

- קטע קצר ("קל") הוא קטע אשר $d_i < c$. לא עושים במקרה זה כלום - סורקים בתוכו בעלות $\mathcal{O}(c)$.
- קטע ארוך ("כבד") הוא קטע אשר $d_i \geq c$. בקטע כזה אנחנו נשמור במפורש את כל t המיקומים של האחדות בתוכו.

השאילתא, היא בעלות $\mathcal{O}(1)$ כי אין כאן עבודה - שולפים מהעבודה המוקדמת.

נשים לב שיש לכל היותר $\frac{n}{c}$ קטעים מאורך c ומעלה. לכל אחד שומרים t מיקומים של $\log_2(n)$ ביטים. לכן החסם כאן לזיכרון הוא $\frac{n}{c}\cdot t\log_2(n)$.

כעת, נגדיר $t=\log_2(n)$ וכן $c=\log_2^2(n)$. אזי סיבוכיות המקום הינה

$$\mathcal{O}\left(\frac{n}{t}\cdot\log_2(n)\right)+\mathcal{O}\left(\frac{n}{c}\cdot t\log_2(n)\right)=\mathcal{O}(n+n)=\mathcal{O}(n)$$

וכן - עלות שאילתא היא סה"כ $\mathcal{O}\left(\log_2^2(n)\right)$. העיבוד המקדים לינארי.

1.2.4 פתרון רביעי

מה נעשה כרגע? הקטעים הארוכים - מטופלים בדיוק כמו קודם. ההבדל הוא בקטעים הקצרים. במקום לסרוק אותם, מה שיעלה $\mathcal{O}\left(\log^2(n)\right)$ נשים לב כי כל קטע קצר הוא בעצמו מערך ביטים קטן. באורך פחות מ$c=\log_2^2(n)$. לכן אפשר לבנות את אותו מבנה עליו בדיוק.

התחכום שיגרום לזה לחסוך מקום - בתוך קטע קצר באורך m מספיק לשמור מיקומים יחסיים לתחילת הקטע. כל מיקום כזה דורש $\log(m)$ ביטים במקום $\log(n)$. מכאן יבוא השיפור.

$\log^*(n)$ הוא מס' הפעמים שניתן להפעיל $\log$ על n עד שנגיע ל1.

אם כן, נסתכל על תת בעיה באורך m ונבחר בה $t=\log(m)$ ו$c=\log^2(m)$. סה"כ נקבל לפי אותו חישוב $2m$ ביטים למיקום. תתי הבעיות באותה רמה זרות, לכן כל רמה עולה $\mathcal{O}(n)$ ביטים לכל היותר.

מספר הרמות: נשים לב שהגדלים יורדים - $n \to \log^2(n) \to \log^4(n) \to \ldots$ ולכן יש סה"כ $\mathcal{O}(\log^*(n))$ רמות. לכן סה"כ סיבוכיות המקום $\mathcal{O}(n\log^*(n))$ ביטים.

העיבוד המקדים - כל רמה היא מעבר לינארי על קטעיה, לכן סה"כ $\mathcal{O}(n\log^*(n))$

שאילתא - בכל רמה $\mathcal{O}(1)$ עבודה לשליפת הערך, ואז התאמה בחישוב פשוט. יש $\mathcal{O}(\log^*(n))$ רמות ולכן זמן השאילתא $\mathcal{O}(\log^*(n))$.

1.3 Range Minimum Query (RMQ)

נגדיר את הבעיה הבאה:

הגדרה 12. נתון מערך בינארי $A \in \mathbb{R}^n$. בהינתן $i,j \in [n]$ יש למצוא את:

$$x = \arg\min_{i \le k \le j} A[k]$$

המערך מועבד מראש. כל שאילתא מקבלת שני אינדקסים $i \le j$ ומבקשת את המיקום של הערך הקטן ביותר בטווח $A[i...j]$.

1.3.1 פתרון ראשון

בשאילתא פשוט נסרוק את האיברים בתחום, נשמור את המיקום של הקטן ביותר עד כה. אם הגיע קטן יותר ־ נחליף. סיבוכיות השאילתא היא

$$\mathcal{O}(j-i) = \mathcal{O}(n)$$

אין סיבוכיות מקום.

1.3.2 פתרון שני

ניצור טבלה של כל הזוגות. נשמור את התשובה לכל זוג (i,j). יש כ$\mathcal{O}\left(n^2\right)$ זוגות וכל תשובה היא אינדקס של $\mathcal{O}(\log_2(n))$ ביטים. לכן סה"כ סיבוכיות המקום היא $\mathcal{O}\left(n^2 \log_2(n)\right)$. השאילתא ־ קריאה מהטבלה בעלות $\mathcal{O}(1)$. את הטבלה נמלא בתכנות דינמי פשוט:

$$M[i,j] = \begin{cases} i & i = j \\ j & A[j] \le A[M[i,j-1]] \\ M[i,j-1] & o.w \end{cases}$$

1.3.3 פתרון שלישי

כעת, לכל מיקום i במערך נשמור את המיקום של המינימום בקטעים שמתחילים בi ואורכם חזקה של 2: $1,2,4,8,...$ ימינה מi, ואותו דבר שמאלה מi. ישנם $\log_2(n)$ קטעים כאלו לכל היותר. כלומר אם נסביר ברור יותר ־ אנחנו שומרים את מיקום המינימום רק עבור אורכים בחזקה של 2. לכל מיקום i במערך שומרים:

- את המינימום בקטע שמתחיל בi והולך ימינה באורך $2,4,8,16,..$
- את המינימום בקטע שמתחיל בi והולך שמאלה באורך $2,4,8,16..$

כמה קטעים כאלו יש? יש $\log_2(n)$ קטעים לכל כיוון. לכן מהו הזיכרון סה"כ?
עושים זאת לכל מיקום i, ויש n כאלו לכן:

$$n \cdot 2\log_2(n) \cdot \log_2(n) = \mathcal{O}\left(n \log_2^2(n)\right)$$

נשים לב כי הלוג האחרון כאן הוא משמירה של ערך שיתכן ומגיע עד לn. מיוצג ע"י $\log_2(n)$ ביטים.

מבצעים עיבוד מקדים: קטע באורך 2^k שמתחיל בi הוא בדיוק שני קטעים צמודים באורך 2^{k-1}: אחד מתחיל בi ואחד ב$i+2^{k-1}$. את שניכם חישבנו קודם, לכן :

$$M[i][k] = \min\left\{M[i][k-1], M\left[i+2^{k-1}, k-1\right]\right\}$$

הסבר: כל תא מחושב ב$\mathcal{O}(1)$ ויש $\mathcal{O}(n \log n)$ תאים לכן זמן עיבוד המקדים הוא $\mathcal{O}(n \log n)$.

מה עם זמן שאילתא? מקבלים i,j. אורך הקטע $L = j - i + 1$.
1. מוצאים את החזקה הגדולה ביותר של 2 שלא עולה על L: $2^k \le L \le 2^{k+1}$.
2. לוקחים את הקטע באורך 2^k שמתחיל בi והולך ימינה.

3. לוקחים את הקטע באורך 2^k שמתחיל בj והולך שמאלה.
4. כל קטע כזה הוא באורך לפחות $\frac{L}{2}$ ולכן שניהם יחד מכסים את כל $[i,j]$. הם חופפים באמצע - וזה לא בעייתי.
5. מחזירים את הקטן מבין התשובות השמורות.

למה חפיפה לא מפריעה? כי מינימום לא נפגע מספירה כפולה.

דוגמה: אם $j=12, i=3$ אזי $L=10$ ולכן $k=3$ כי $2^3=8$.
הקטע מ3 באורך 8 ימינה הוא $[3,....,10]$. הקטע מ12 באורך 8 שמאלה הוא $[5,....,12]$
יחד הם מכסים את:

$$[3,...,10] \cup [5,...,12] = [3,....,12] = A\,[i,...,j]$$

סה"כ זמן השאילתא הוא $\mathcal{O}(1)$. נעיר כי דורש חישוב $K=\log(L)$ אבל נניח שזה נעשה בפקודת מעבד פשוטה.

1.3.4 פתרון רביעי

הרעיון כזה - נחלק את המערך לבלוקים באורך t. מבנה החזקות של פתרון השלישי נשמר עבור כל בלוק. כלומר במיקומים $t, 2t, 3t, ..$. נקרא לנקודות אלו נקודות שמורות. נשים לב כי יש לנו כעת $\frac{n}{t}$ מיקומים. לכן גודל הזיכרון שנתפס:

$$\frac{n}{t} \cdot 2\log(n) \cdot \log(n) = \frac{2n}{t}\log^2(n)$$

שאילתה בסיסית: i וj לא יושבים לרוב על נקודות שמורות. לכן מפצלים את הקטע לשלושה קטעים:
1. מi עד לנקודה השמורה הבאה. פחות מt איברים.
2. מהנקודה השמורה הראשונה ועד האחרונה באותו קטע.
3. מהנקודה האחרונה באותו קטע עד לj. פחות מt איברים.

לכן סה"כ זמן השאילתא $\mathcal{O}(t)$ - שמירה בקצוות.
עיבוד מקדים: מחשבים את המינימום של כל בלוק בזמן לינארי, סה"כ $\mathcal{O}(n)$. ואז בונים את טבלת החזקות על $\frac{n}{t}$ הנקודות ב$\mathcal{O}\left(\frac{n}{t}\log(n)\right)$.
נבחר $t=\sqrt{n}$ ואז שאילתא $\mathcal{O}(\sqrt{n})$, הזיכרון $2\sqrt{n}\log^2(n)$ ועיבוד מקדים בזמן $\mathcal{O}(n)$.
0>0